计算机操作系统

第6章 文件系统(1)

文件系统概述

- 文件系统：操作系统中实现对数据进行有效管理的模块
- 文件系统的引入
  - 主存储器只能临时保存数据，外存储器可以长期保存数据
  - 主存储器容量较小，外存储器容量大
  - 外存储器的存储介质种类繁多，不同的存储介质，物理特性差异很大
  - 发挥数据的信息作用

文件系统概述

- 文件及按名存取
- 文件分类
  - 按文件的性质分为：
    - 系统文件——系统软件对应的文件
    - 库文件——已编译的，可供其他程序使用的子程序文件
    - 用户文件——由用户提供的文件
  - 按文件的存取控制分为：只读文件、读写文件、可执行文件、隐藏文件 等
  - 按文件的组织（在文件系统中的作用）分为：
    - 普通文件——用于实际地存储数据
    - 目录文件——用于表示目录结构
    - 设备文件——用于表示设备

文件系统概述

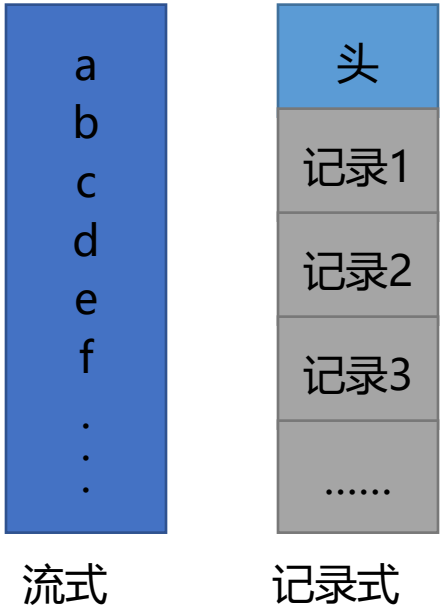
- 按文件内容的表示分为：
  - 文本文件——存储能被用户直接理解的文本
  - 二进制文件——存储不能被直接理解的二进制数据
- 按文件内容的组织形式分为：
  - 逻辑结构
  - 物理结构

文件系统概述

- 一个有效的文件系统，应当具备的功能有：
  - 文件内容的组织——逻辑结构与物理结构
  - 文件与目录管理
  - 存储空间管理
  - 文件系统的命令接口与程序接口
  - 文件的共享与安全性保障
- 常见的文件系统
  - DOS系统的FAT16文件系统
  - Windows系统的FAT32和NTFS
  - Linux系统和ext2等

文件的逻辑结构

- 文件的逻辑结构指文件内容的组织形式，侧重于方便用户使用。
- 按逻辑结构（用户访问方式）分为
  - 流式文件——文件可被看作一串有序的字符流，内部无结构划分
  - 记录式文件——文件内的数据被划分为具有逻辑完整性的单元，每个单元称作一条记录，每条记录可以包含若干个数据项（字段）

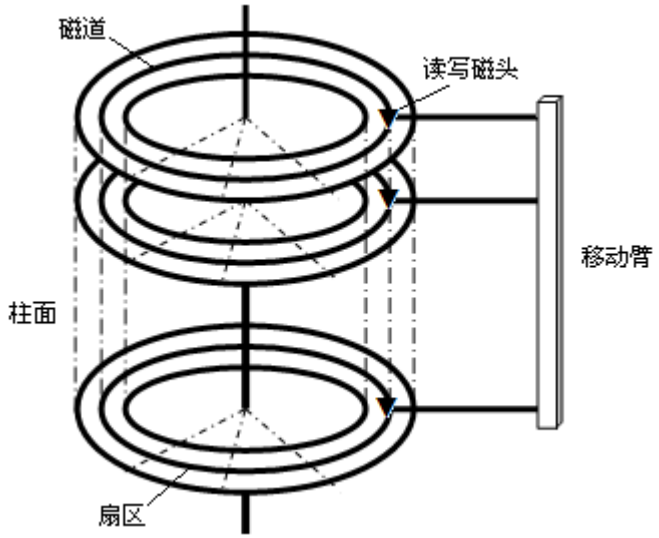


文件的物理结构

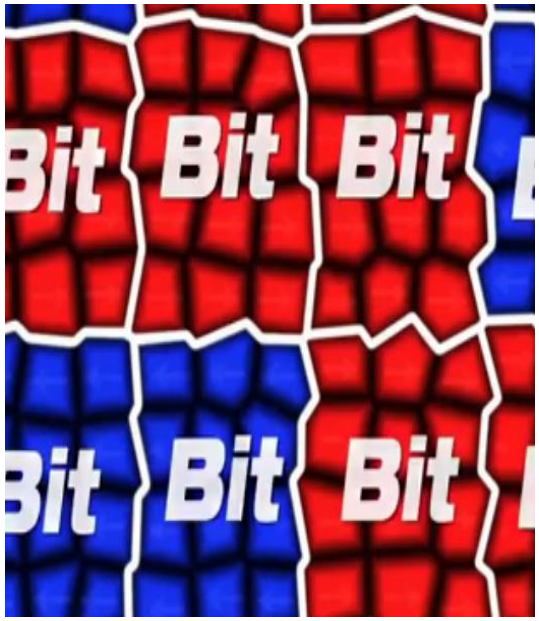
- 文件的物理结构指文件内容在存储介质上的组织形式，对用户而言是透明的，侧重于提高存储空间的利用率和存取速度。
- 外存储设备按其访问特性可分为：
  - 顺序存取设备——只能按照数据在设备上的顺序依次访问数据
  - 随机存取设备——能够在设备上的任意位置存取数据
- 文件的存储介质
  - 磁盘
  - 磁带
- 物理结构分类
  - 连续
  - 链接
  - 索引

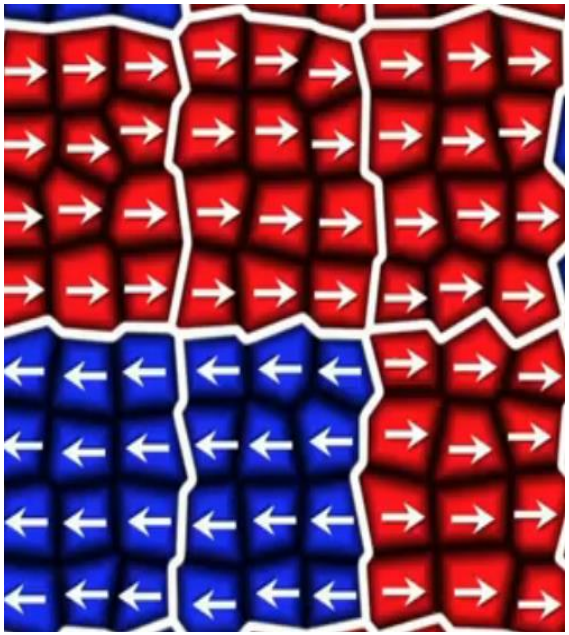
磁盘

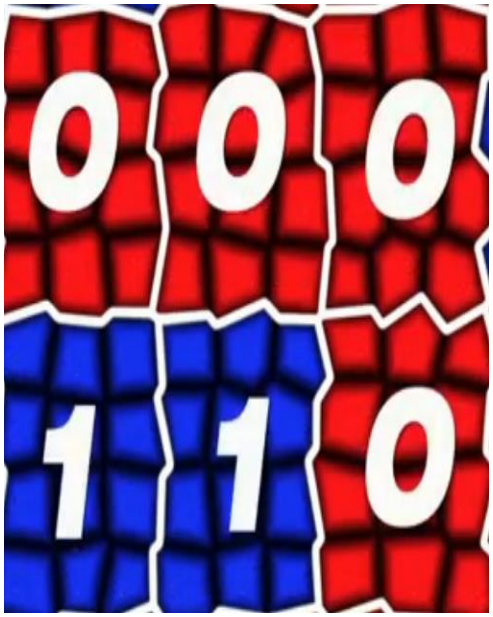
- 一块磁盘中包含一组圆形盘片，各自独立的磁化物质（存储单元）分布于这些盘片上（单面或双面），以其磁化方向来表示一位二进制数。
- 对磁盘的访问，可以通过指定存储单元的位置进行随机存取。
- 用于表示存储单元位置的相关术语：
  - 面或磁头（H）
  - 磁道（T）与柱面（C）
  - 扇区（S）

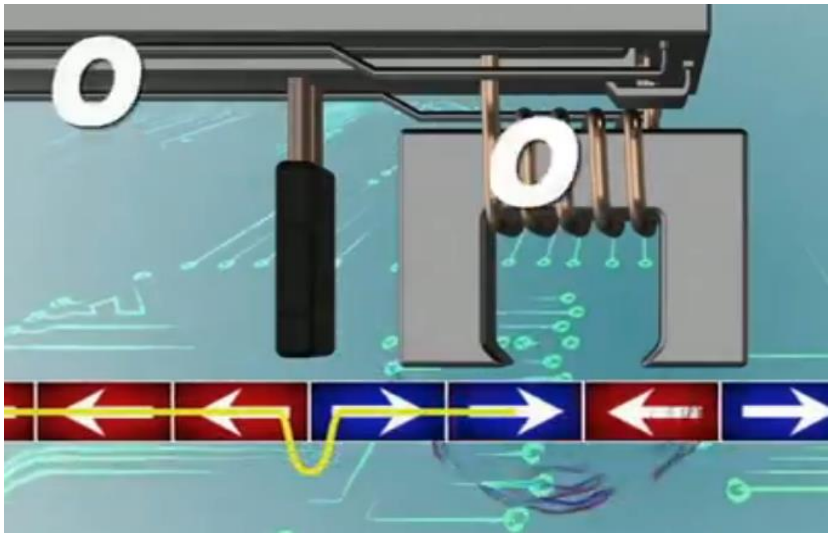


磁盘



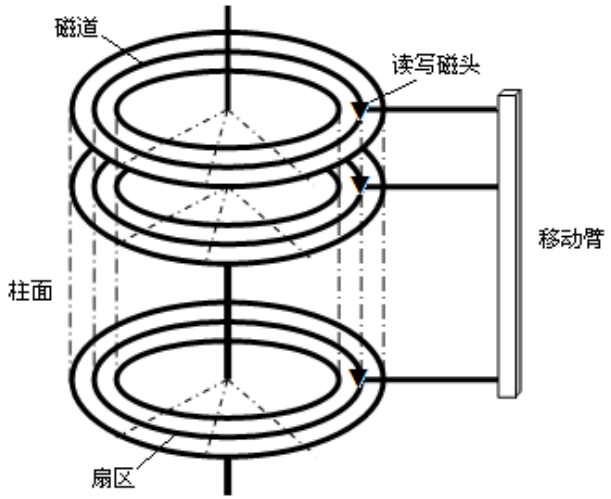






物理地址参数

- 用三元组“柱面号-磁头号-扇区号”为坐标指示磁盘上的一个块（也称扇区，其中包含一组存储单元），此三元组称作这个块（扇区）的物理地址参数（CHS）。
- 设某磁盘磁头数量为heads，扇区个数为sectors，则
  - 一个柱面的存储量为heads×sectors个扇区（物理块）
  - 一个磁道的存储量为sectors个扇区（物理块）。



逻辑地址(块号)

- 一个扇区的物理地址参数（CHS）与逻辑地址（块号）的关系：
  - 设CHS为（c0，h0，s0）的块（扇区）对应块号block0，则CHS为（c，h，s）的块（扇区）对应块号：
    - block = heads×sectors×(c-c0) + sectors×(h-h0) + s-s0+block0
  - 块号为block的块（扇区）的CHS为：
    - s = (block - block0) % sectors + s0
    - h = ((block - block0) / sectors) % heads+ h0
    - c = ((block - block0) / sectors) / heads + c0

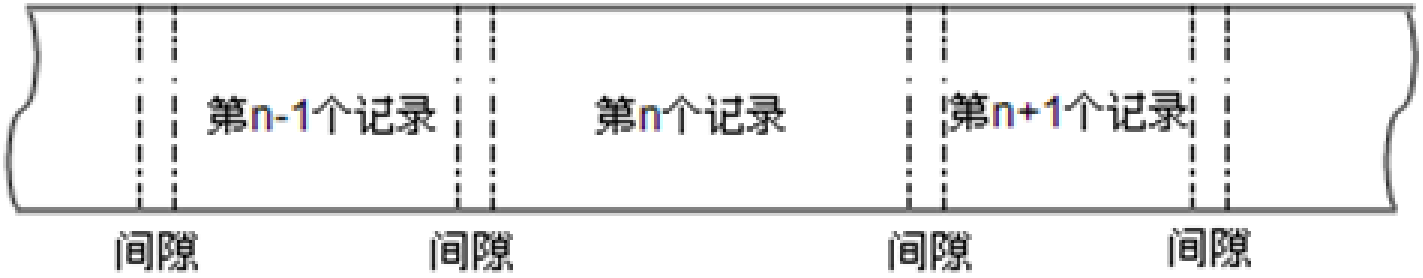
逻辑地址(块号)

- 若某文件系统使用n位的块号，则可以表示2^n个扇区（块）。
- 外存储设备最大存储容量不可超过2^n×块容量。

- 对于较大的外存储设备，文件系统内部将2^k个扇区划分为一个簇，以簇（而非扇区）作为基本IO操作单元，并为其编排逻辑地址（块号）。
- 外存储设备存储容量限制：2^n ×块容量 = 2^{n+k} ×扇区容量。

磁带

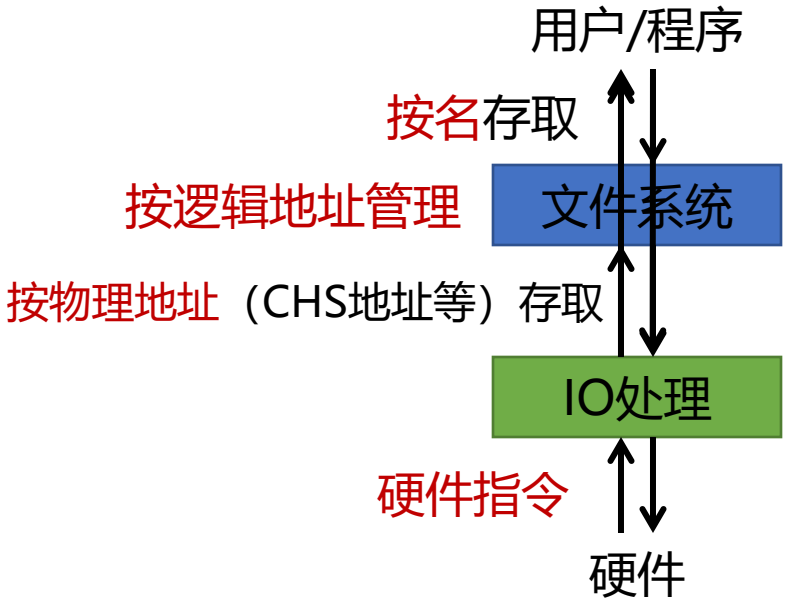
- 磁带上各自独立的磁化物质（存储单元）分布于一条形的承载介质上，以其磁化方向来表示一位二进制数。
- 磁带上不同的数据段之间用特殊编码的间隙隔开。



- 磁带并不支持按地址访问数据，只能够依次访问数据，并根据期间跨越的间隙个数来确定一个数据段的位置。因此它是典型的顺序存取设备。

文件IO操作

- 在从外存指定位置读写文件内容时，CPU组织相关的存取参数，然后通过IO中断启动存储设备的驱动器，由设备的驱动器完成具体的读写操作，把文件的这种输入/输出操作，称为文件IO操作。
- 文件IO操作的基本单位是物理块。
- 操作系统封装IO操作，并向用户提供文件系统。
- 这样做的目的是：
  - 向用户隐藏各种外存储设备的特性差异，提供统一的访问方式
  - 便于实现信息的存储、加工、共享等管理行为



物理结构分类

- 文件的物理结构分为：
  - 1. 连续结构

    文件中的所有记录按顺序依次存放在相邻的物理块内。

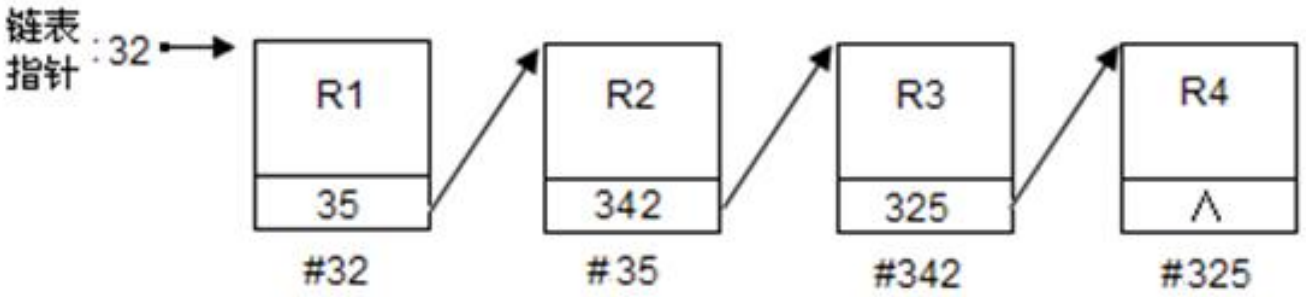
    若采用连续结构，则对于某文件，系统需记录其首块的块号及块数。

    - 管理简单
    - 存取速度快（随机存取）
    - 需要连续空间，降低了存储器利用率
    - 不利于文件的修改（增删记录）

物理结构分类

➢ 2. 链接结构

- 借鉴链表的数据结构，将文件分散存储在多个物理块中，在块中指定一部分存储单元用于指示文件下一个块的位置（指针）。



- 若采用链接结构，则对于某文件，系统需记录其首块的块号。

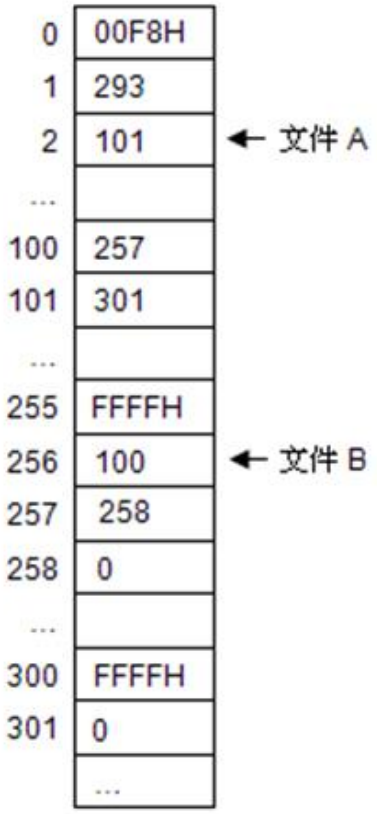
物理结构分类

- 2. 链接结构
  - 链接结构的特点：
    - ✓无需连续空间，提高存储器利用率
    - ✓方便文件修改
    - ✓无法直接定位文件中的位置，只能顺序查找（顺序存取）
    - ✓可靠性无法保证
    - ✓破坏了信息完整性

物理结构分类

➢ 2. 链接结构

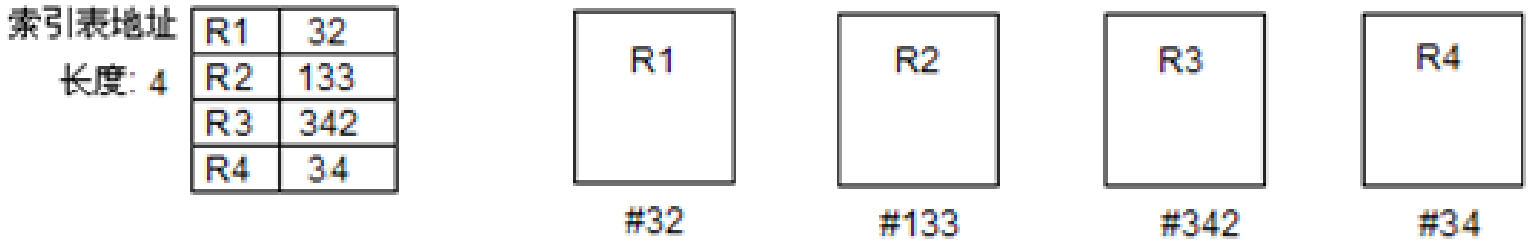
- FAT系列文件系统中采用改进的链接结构：将指针信息从实际存储数据的物理块中分离，并记录到文件分配表（FAT：File Allocation Table）中。
- FAT中的每一项对应一 个物理块。
  - ✓值FFFF（全1）表示该块空闲
  - ✓值0表示该块为文件尾或损坏
  - ✓其他值指示该块中的文件的下一块位置



FAT16的例子

物理结构分类

- 3. 索引结构
  - 文件的各记录（或部分）分散存放于存储设备的物理块中。利用索引表（类似于页表）记录文件中的记录与物理块的对应关系。



  - 采用索引结构，则对于某文件，系统需记录其索引表位置与长度。

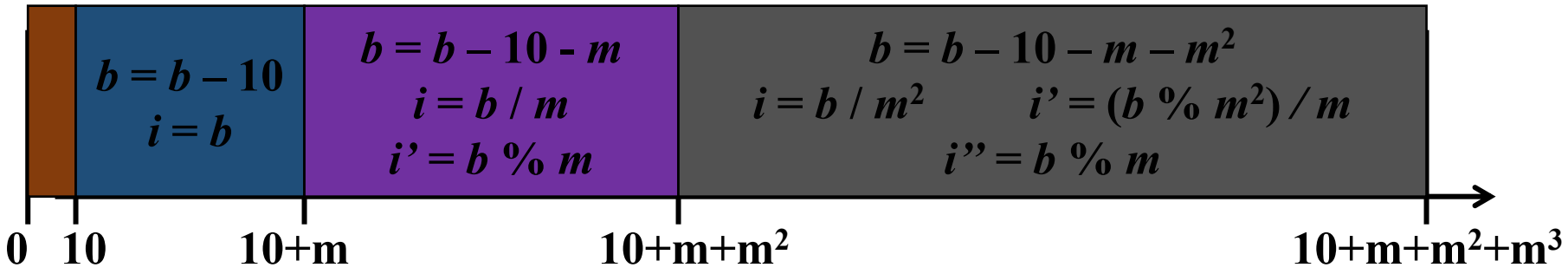
物理结构分类

- 3. 索引结构
  - 索引结构的特点：
    - ✓无需连续空间，提高存储器利用率
    - ✓方便文件的修改
    - ✓可实现随机存取
    - ✓索引表占用额外存储空间
    - ✓为了不连续保存索引表，常用多级索引

      ——检索时的时间开销大

物理结构分类

- 3. 索引结构
  - Unix使用改进的多级索引结构（i-node）
    - ✓直接索引：10个
    - ✓一次间接索引：m个（m为一个块内最多可以存放的索引数）
    - ✓二次间接索引：m^2个
    - ✓三次间接索引：m^3个



物理结构分类

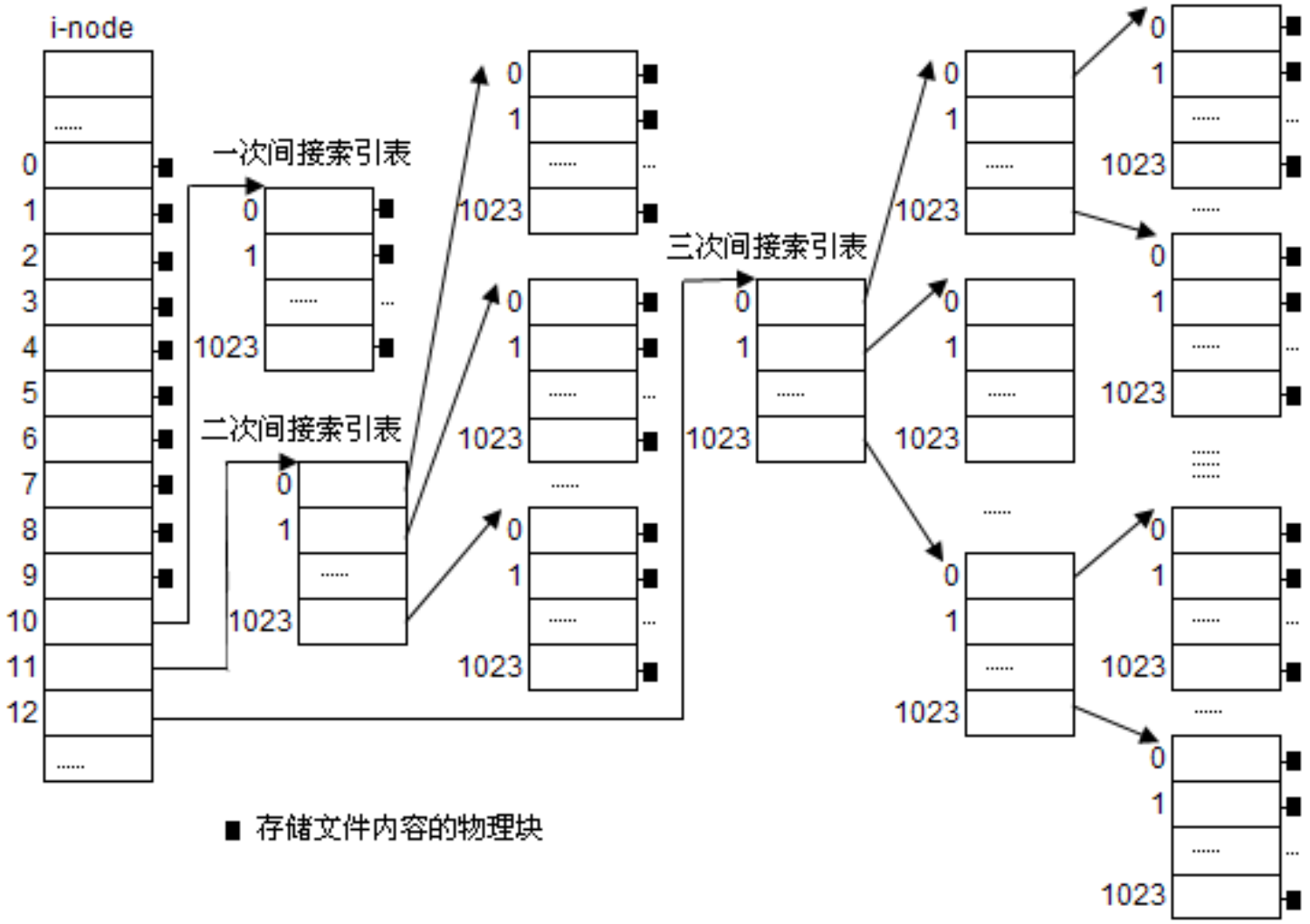


图6-8 UNIX的多级索引结构

作业

4. 假定 UNIX 系统下的某文件 i-node 结构及部分索引表如下图所示，若磁盘的块大小为 1KB，块号用 4 字节表示，试给出该文件内地址 0x80AB 和 0x528E2 所在的磁盘块号和块内地址。

i-node

608
606
56
420
80
316
257
732
91
421
376
577
20

块号：376

0	960
	……
21	711
22	21
23	81
	……
31	422
32	561
33	676
	……

块号：499

0	233
	……
64	455
65	567
	……
74	923
75	777
	……

块号：577

0	499
1	909
	……

块号：909

0	114
	……
64	99
65	232
	……
64	290
65	291
	……

计算机操作系统

第6章 文件系统(2)

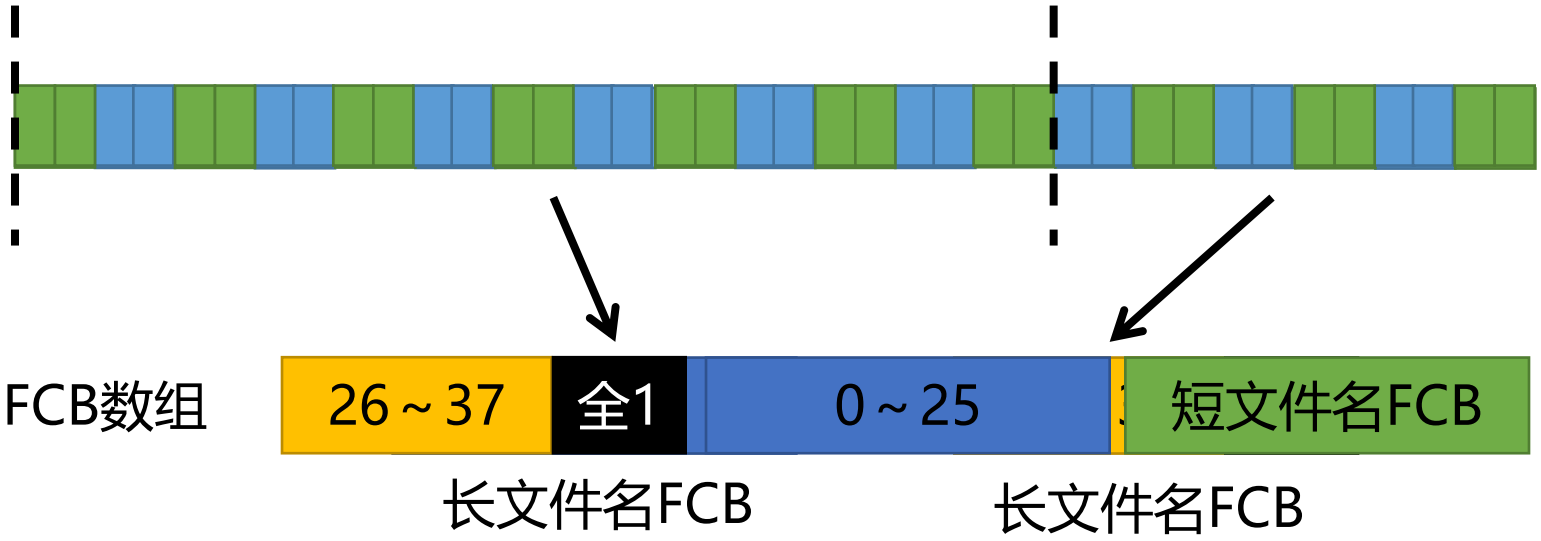
文件控制块(PCB)

- 与进程管理类似，为了记录文件的相关信息，文件系统需要维护特殊的数据结构——文件控制块（FCB），将其存储于外存中。
- 不同的文件系统其FCB设计差异较大，但主要都包含两部分：
  - 描述信息部分——文件名、逻辑结构相关信息、物理结构相关信息
  - 管理信息部分——存取控制信息、使用信息

文件控制块(PCB)

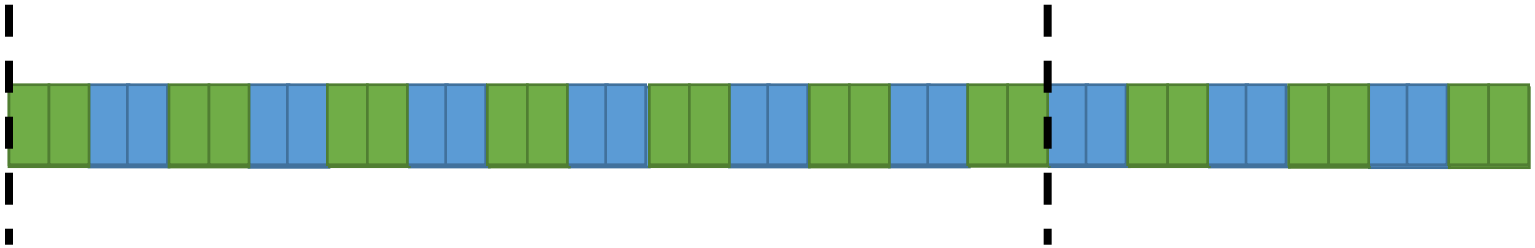
- FAT32的一个FCB的长度固定为32字节。每个文件对应一个FCB数组，此数组中包含1个短文件名FCB和0个或多个长文件名FCB。
  - 短文件名FCB用于记录文件的短文件名（8字符主名，3字符扩展名）和文件的其他相关信息
  - 长文件名FCB仅用于记录文件的全名（当文件具有长文件名时）

文件控制块(PCB)



文件控制块(PCB)

- ✓ 用FCB数组记录长文件名文件时，先将长文件名以Unicode编码（每个Unicode字符占用2个字节）
- ✓ 然后将其按每13个双字节（26个字节，13个Unicode字符）分组，存放进多个长文件名FCB中
- ✓ 将这些长文件名FCB倒序组成FCB数组
- ✓ 在FCB数组的最后添加一个短文件名FCB，其中记录文件的短文件名和其他相关信息

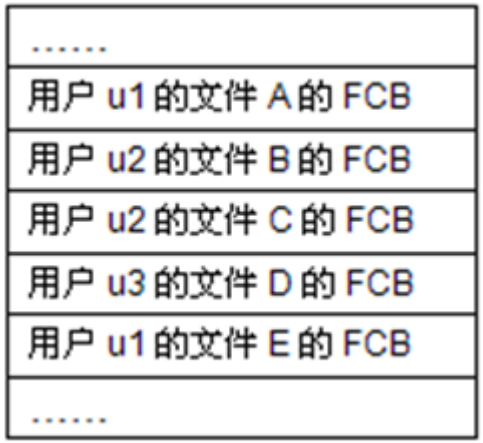


文件控制块(PCB)

- 长文件名文件的短文件名生成规则：
  - 无法识别的字符（非ASCII字符）用“_”替代
  - 取全名的前6个字符加“~1”作主名，扩展名的前3个字符作扩展名
  - 若重名，则编号“~1”依次增加，最多编号到“~4”
  - 若编号超过“~4”，则保留主名的前2个字符，用全名剩下部分的头4个字符生成其他4个字符，并从“~1”开始重新编号

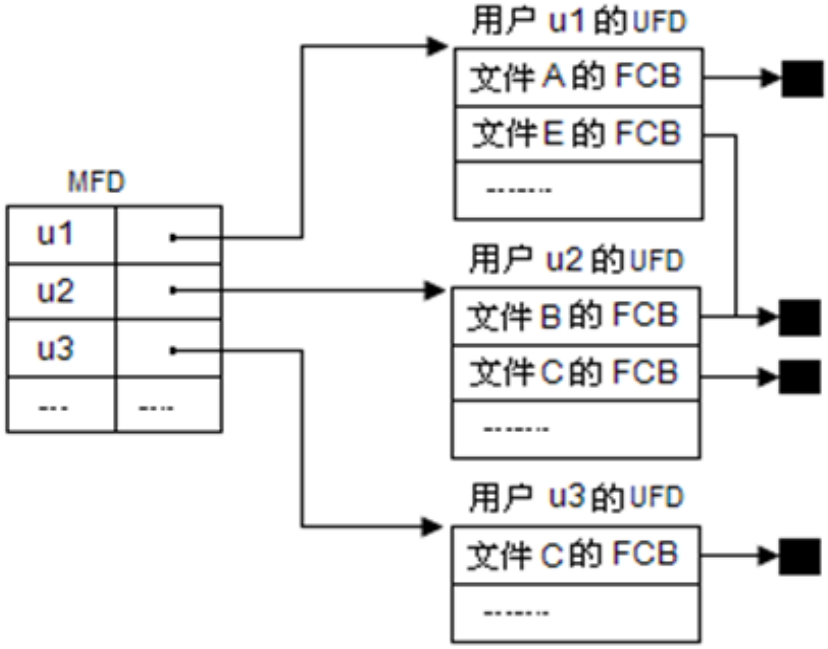
文件目录及其结构

- 若干个文件的FCB数据的有序集合称作文件目录，文件目录本身也以文件形式保存于外存中，称作目录文件。
- 如何划分文件，把若干文件组织在一起作为一个文件目录，就导致目录有不同的组织形式，目录的组织形式称为目录结构，有三种目录结构：单级、二级、多级。
- 单级目录结构：若将文件系统中的所有文件的FCB都保存在一个目录中，称作单级目录结构。
  - 管理简单，易于实现
  - 任何两个文件都不能重名
  - 访问文件时，检索范围大，影响了存取时间



文件目录及其结构

- 二级目录结构中存在一个系统主目录（MFD）和多个用户文件目录（UFD）。MFD中存放用户名与对应的用户文件目录（UFD）的存储起始地址，UFD中存放用户文件的FCB。
  - 结构简单、容易实现
  - 一定程度上减少了检索的开销
  - 解决不同用户之间的重名问题
  - 容易实现用户之间的文件共享
  - 拥有多文件的UFD检索开销大
  - 同个UFD下文件不可重名



文件目录及其结构

- 多级目录结构（目前常见的），是在二级目录结构基础上进一步扩展：任意目录文件中都可以存放目录的FCB或文件的FCB，以形成多级目录。
  - 根目录(Root)
  - 父目录/子目录
  - 绝对路径(Absolute Path)
  - 当前目录(Current Directory)
  - 相对路径(Relative Path)
  - 两个特殊的子目录：“.”和“..”

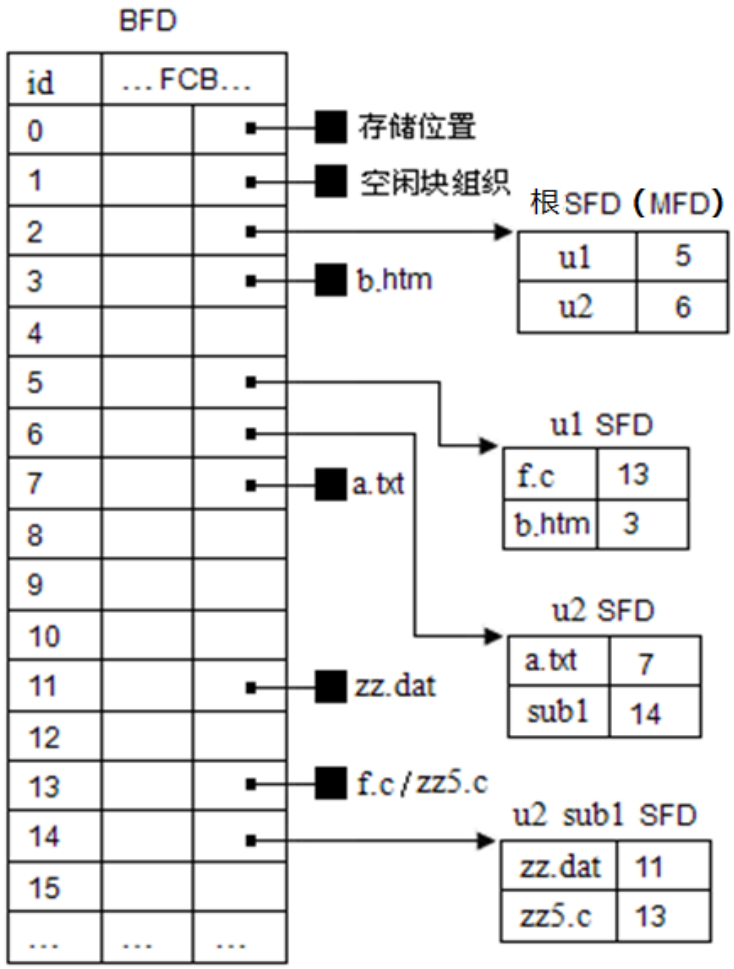


文件目录及其结构

- 目录文件中直接存放FCB引起的问题：目录文件本身较大，搜索文件时IO操作过多。
- 解决思路：将文件名信息从FCB中抽离出来。
- Unix系统采用目录项分解法：所谓的目录项，就是文件目录的各个FCB数据，将文件目录分解为SFD和BFD，每个文件或目录拥有唯一的标识：
  - 基本文件目录BFD：记录文件标识和除文件名外的FCB数据
  - 系统主目录MFD：记录用户及其第一级符号文件目录的标识
  - 符号文件目录SFD（按多级目录文件组织）：记录文件名和文件标识的对应关系

文件目录及其结构

- 进入一个目录时实际上打开（读入）一个SFD。访问文件或子目录时按名搜索，得到标识后再查找BFD，以找到对应的FCB，从而访问需要的文件或子目录。
  - 搜索文件或子目录的开销减小
  - （无搜索的）文件访问开销增加



文件存储空间管理

为了记录外存储器中存储空间的使用情况，操作系统可以采用：

- 位示图法
- 空闲块表法

首块块号	连续块数
2	1
4	1
9	2

缺陷：连续空闲块个数受限制

- 空闲块链表法



缺陷：只能逐块分配；分配和回收空闲块时需两次外存访问

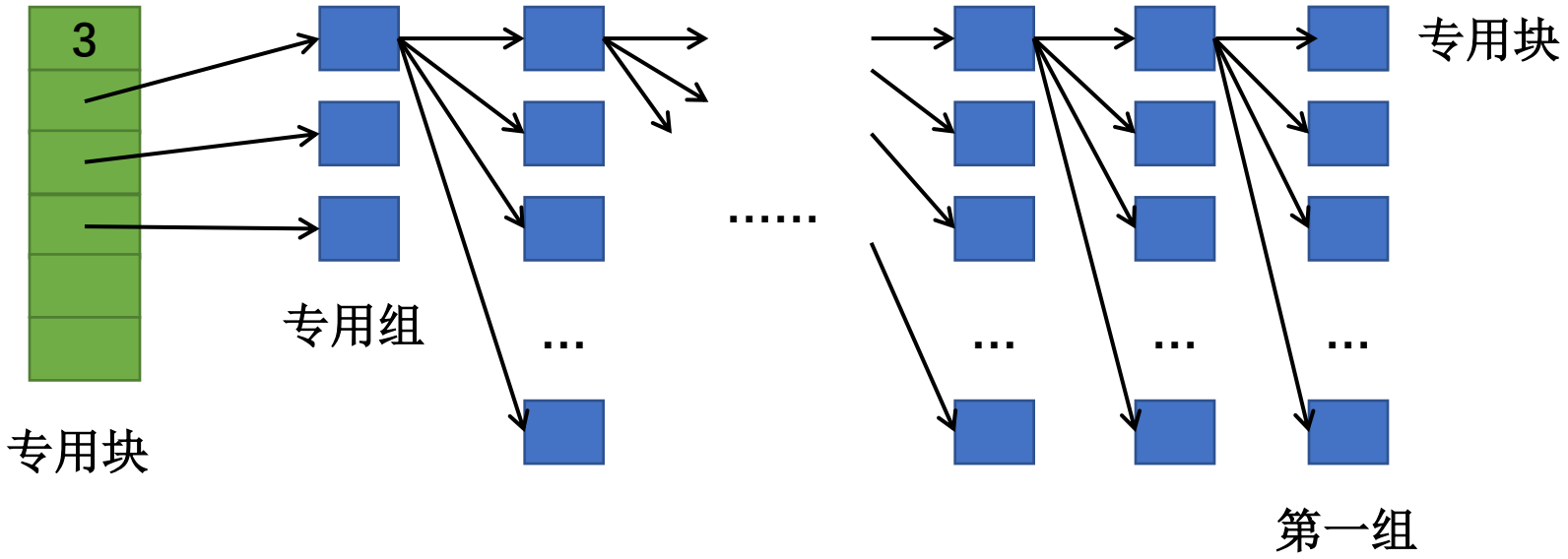
文件存储空间管理

Unix系统采用改进的空闲块链表法——空闲块成组链接法。

将系统的n块作为一组，将每组的块数和块号存入第一组的前一块中。

最后一组的块数和块号存入超级块的栈中。

n=0表示已无空闲块。

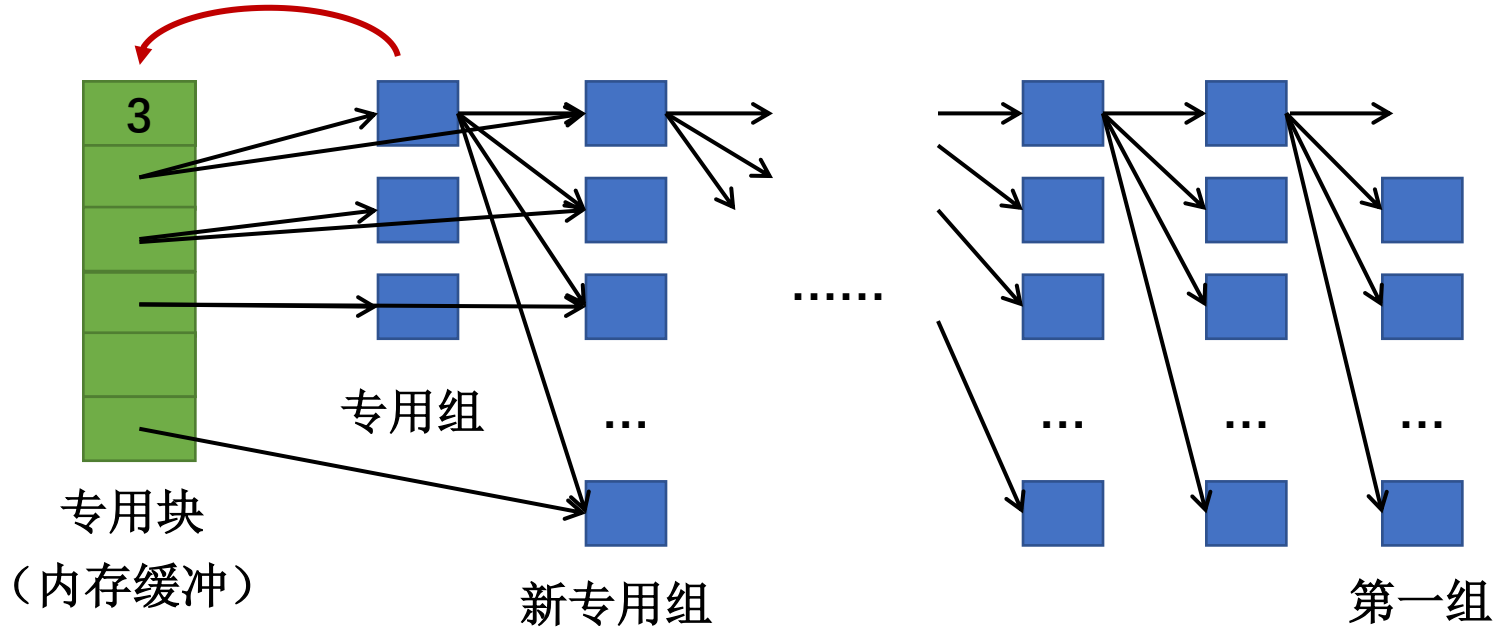


文件存储空间管理

Unix系统采用改进的空闲块链表法——空闲块成组链接法。

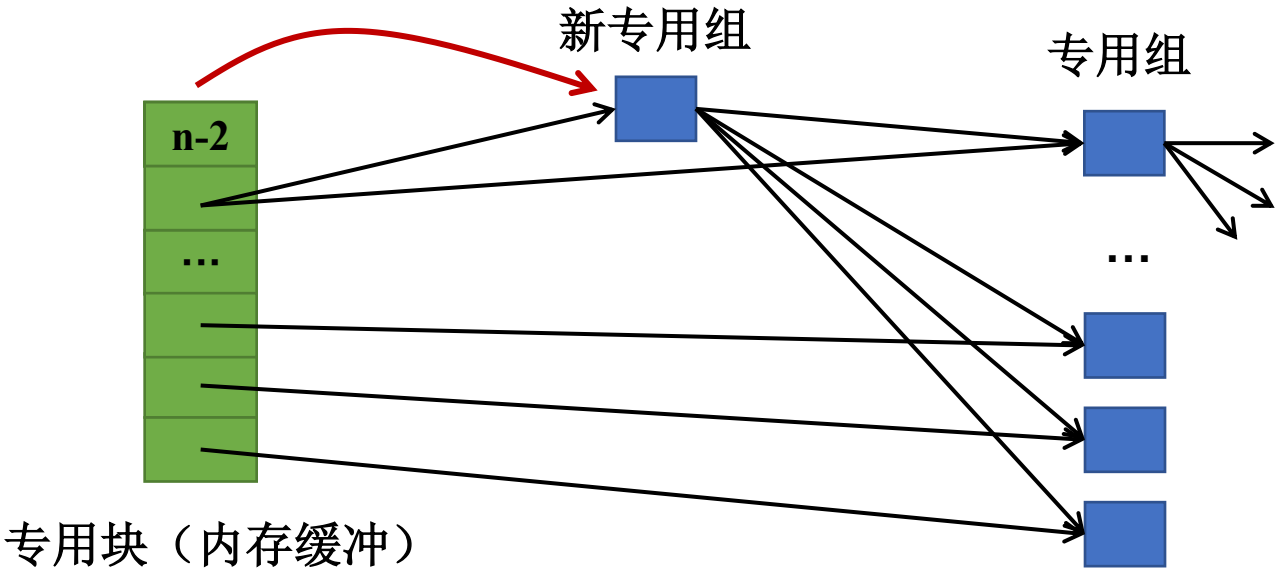
- ✓ 文件系统建立时（空外存），将所有空闲块分组（每组n个块，n=100）。
- ✓ 其中，最后一组可能不满n块，而第一组实际的空闲块为n-1块（第一块被作为专用块，用于存储相关信息）。
- ✓ 在每组的第一个块中记录上一组的块数和块号（指针）。记录中第一组的块数仍记n，但专用块的块号被记作0。
- ✓ 最后一组的块数和块号将存在专用块中，称这组为专用组。

文件存储空间管理



文件存储空间管理

需要用组的块数超过用时，分配空间内的指针信息复制到新闲块的块号记录到闲块成为新的增加组用组的块中。



空闲块成组链接法使得在大部分情况下只需要进行一次对专用块（或缓存）的访问即可完成空闲块的分配和回收。

文件存储空间管理

- ✓ 需要分配存储空间时，查找专用块中的记录，从而找到专用组的最后一个块，将其分配，并减少专用组的块数记录。
- ✓ 若将被分配的是某组的第一个块（分配后此组将消失），则分配之前需要将其中的指针信息复制到专用块中，使得上一组成为新的专用组。
- ✓ 若根据指针信息指示，将被分配的块的块号为0，说明已无空闲块。
- ✓ 文件系统一般将专用块在内核空间中作缓存，以提高访问速度。但这样做可能由于一些意外断开的情况导致文件记录错误。

- ✓ 需要回收空间时，采用与分配空间相反的方式进行：将新空闲块的块号记录到专用块中，并增加专用组的块数记录。
- ✓ 当专用组的块数超过n时，将专用块内的指针信息复制到新的空闲块中，新空闲块成为新的专用组，并更新专用块中的指针信息。

文件存储空间管理

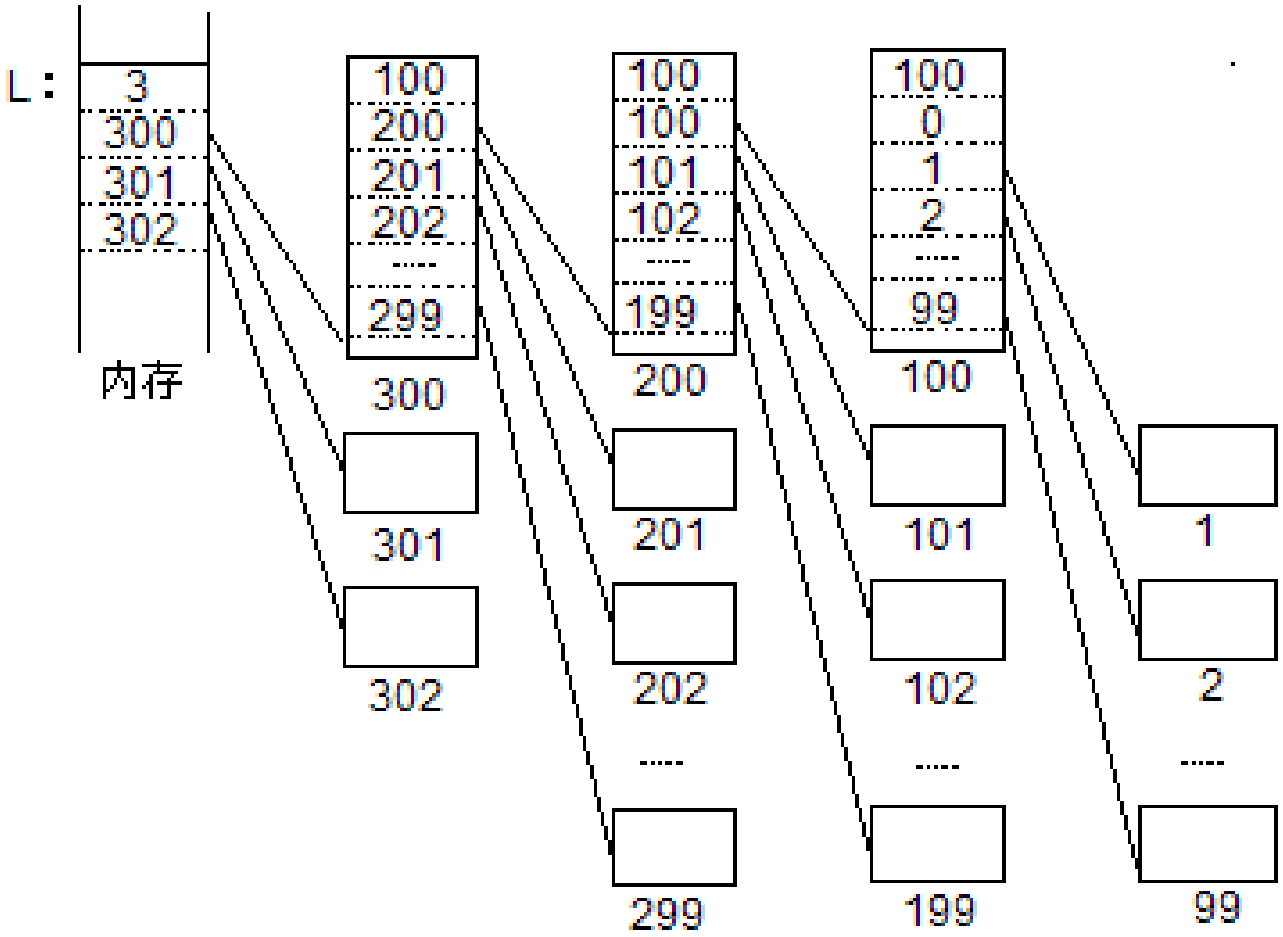


图6-13 空闲块成组链接法例子

文件存储空间管理

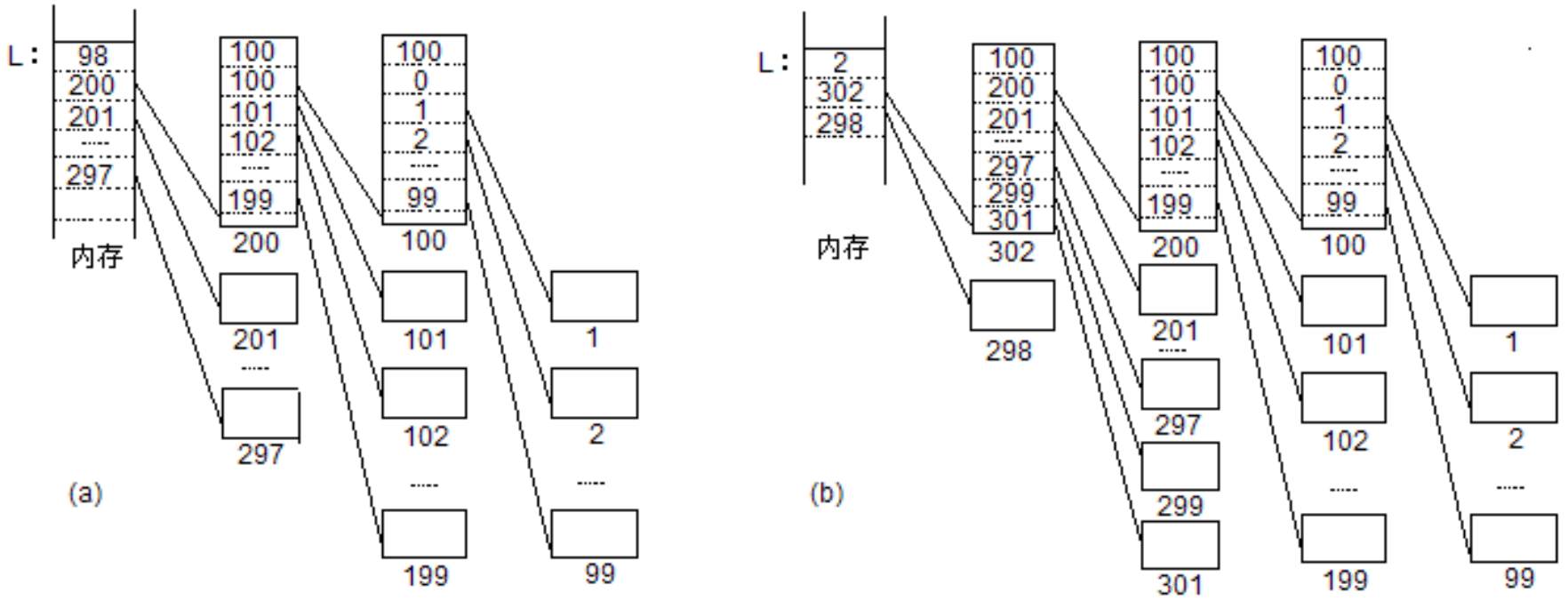


图6-14 空闲块成组链接分配、回收

作业

4. 假定 UNIX 系统下的某文件 i-node 结构及部分索引表如下图所示，若磁盘的块大小为 1KB，块号用 4 字节表示，试给出该文件内地址 0x80AB 和 0x528E2 所在的磁盘块号和块内地址。

i-node

608
606
56
420
80
316
257
732
91
421
376
577
20

块号：376

0	960
	……
21	711
22	21
23	81
	……
31	422
32	561
33	676
	……

块号：499

0	233
	……
64	455
65	567
	……
74	923
75	777
	……

块号：577

0	499
1	909
	……

块号：909

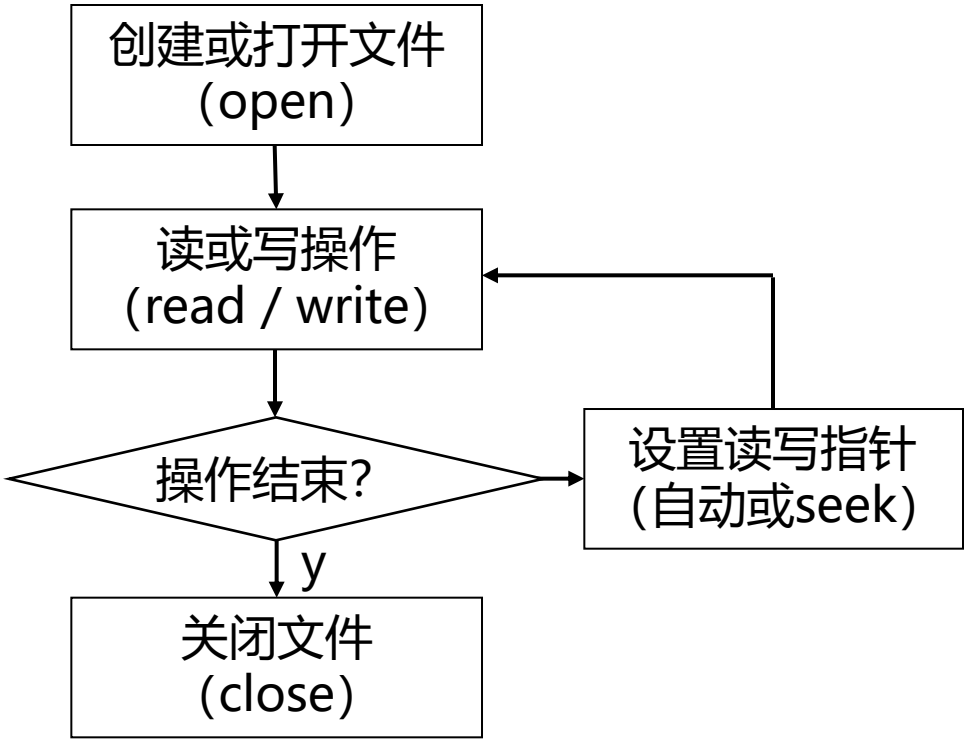
0	114
	……
64	99
65	232
	……
64	290
65	291
	……

计算机操作系统

第6章 文件系统(3)

用户接口

- 用户接口使用户能够操作文件系统。
  - 命令接口使用户能够直接控制文件系统（上机实践部分）
  - 程序接口是一组系统调用，用户程序通过其控制文件系统



文件共享的概念

- 文件共享（File Sharing）指允许不同用户或不同进程访问同一个文件，其目的是：
  - 实现用户或进程之间的信息共享和任务协作
  - 避免因各用户或进程各自保存数据，造成的外存储器额外开销

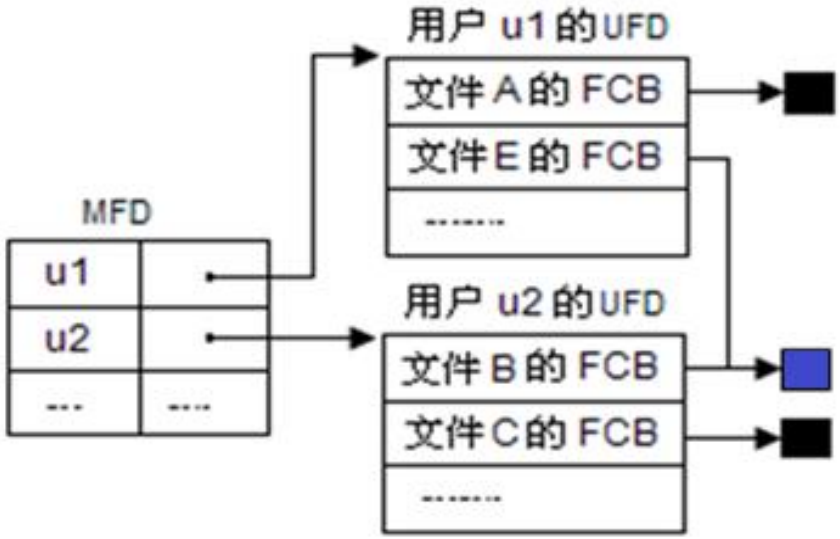
文件共享的方法

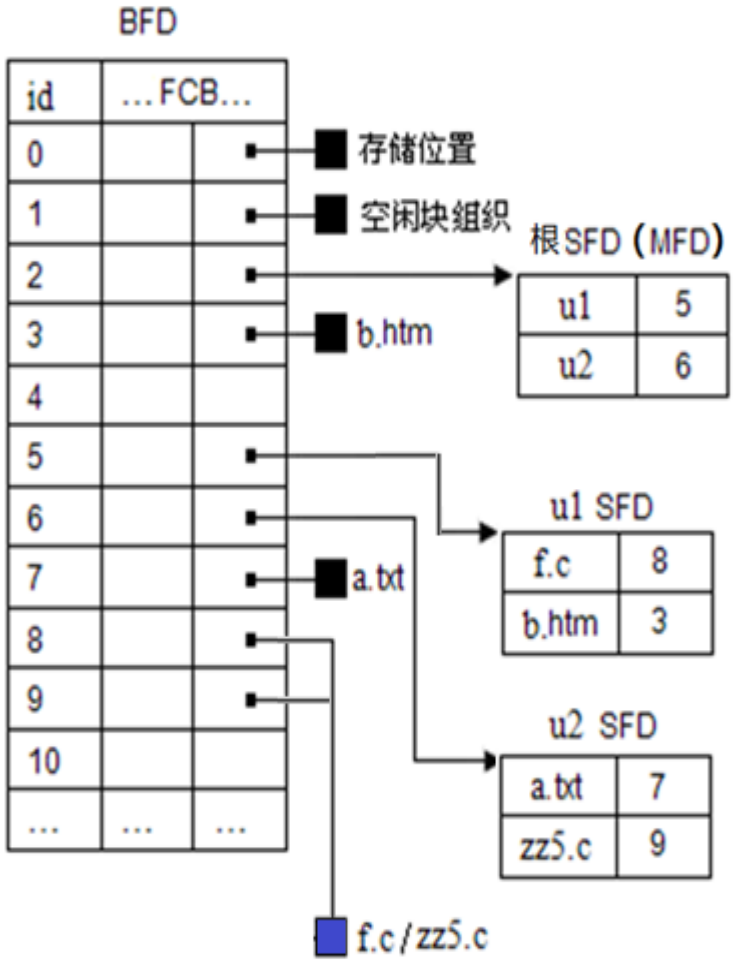
- 文件共享的方法：
- 1. 绕道法
  - 通过公开需要共享的文件的路径，使各用户（进程）能够查找到文件，进而访问文件。
  - 优点：实现简单，直接通过命令接口完成。
  - 缺陷：文件路径必须公开，并只能使用相同的路径与文件名进行访问。

文件共享的方法

❖ 2. 链接法

➢ 通过将不同的FCB指向相同的外存位置，使不同用户（进程）访问到一个实际上相同的文件。



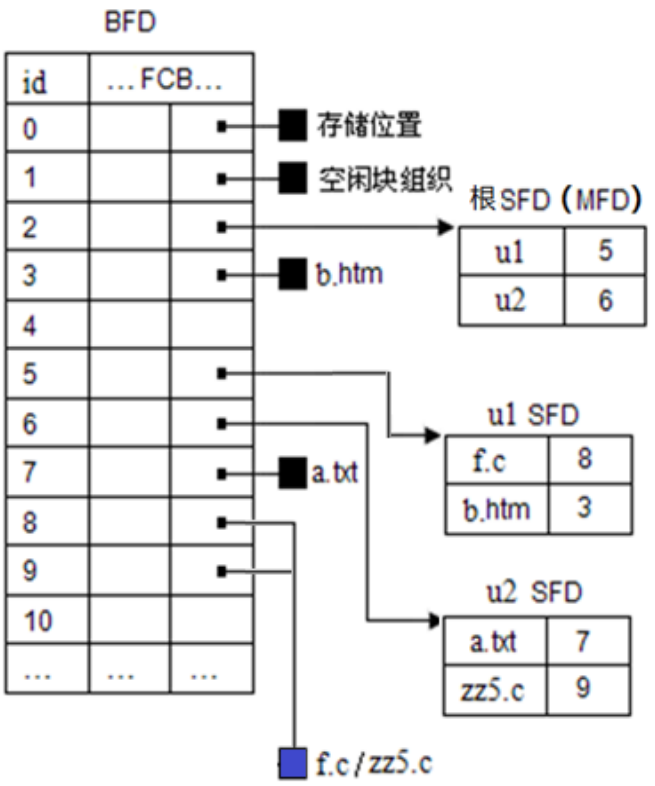


文件共享的方法

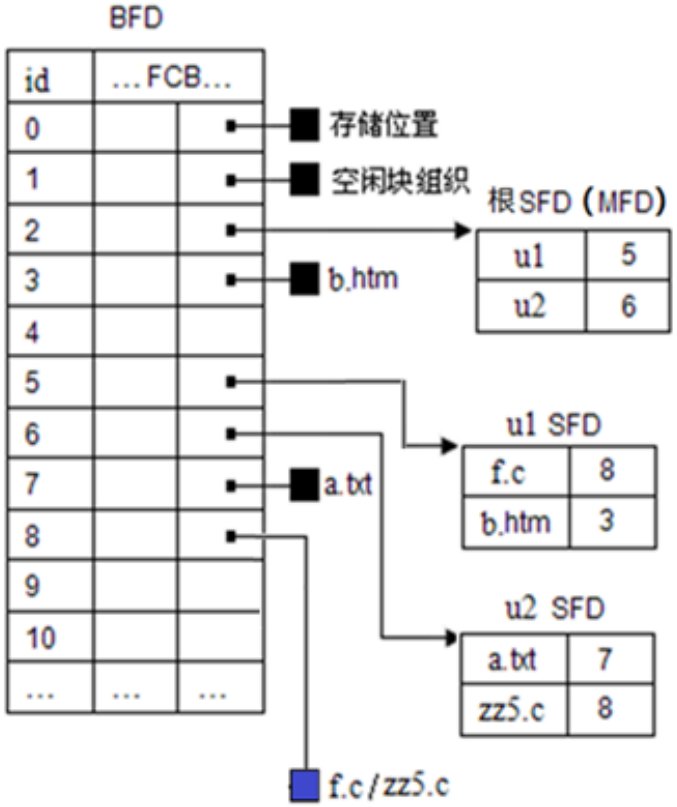
❖ 3. 基本文件目录法：

➢ 在SFD中将一个文件标识对应于不同的访问路径，从而使不同用户（进程）找到实际相同的FCB，从而访问同个文件。

链接法



基本文件目录法



文件共享语义

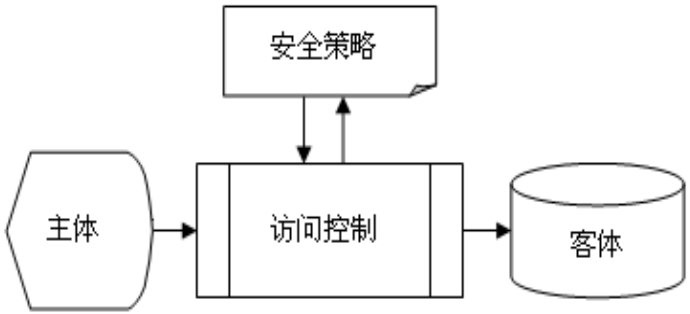
- 当多个用户或进程同时访问一个文件时，对修改（更新）操作的行为作进一步规定，称作文件的共享语义（一致性语义）。
- 三种基本的共享语义：
  - Unix语义：系统按照读写操作的顺序依次执行读写操作。
  - 会话语义：进程打开文件时建立会话，会话期间的修改操作仅对进程自身可见，当文件关闭时结束会话，此时这些修改才对其他进程可见（实际实施）。
  - 永久文件语义：创建文件时可对文件做一次编辑，一旦关闭文件则文件变为只读状态。

文件的保护

- 文件的保护指防止文件内容被破坏，这种破坏包括存储介质的物理损坏导致的文件无法读取和用户误操作导致的文件丢失。
- 文件保护的手段是备份。备份分为：
  - 1. 定期备份：定期将一个外存设备上的数据复制到另一外存设备上。
    - 完全备份——每次备份复制所有数据
    - 增量式备份——每次备份仅复制上次备份以来修改的数据
    - 定期备份需考虑的问题：备份周期的确定、备份保留时间的确定。
  - 2. 活动备份：修改（更新）文件时同时操作多个存储设备上的文件副本，当一个存储设备损坏时可以从其他设备上取得文件。

文件的保密

- 文件的保密指通过访问（存取）控制，防止文件被非法访问。这是文件系统安全保障的核心，以及构建安全操作系统的主要研究内容之一。
- 访问行为的主体是用户和进程，客体是文件，访问行为的类型称作访问属性，主要有读、写和执行。
- 在主/客体间增加访问控制模块，依据访问行为的具体情况（如主体、客体、属性 等）作出允许/禁止访问的决策。
  - 完备性：所有访问均需验证
  - 防护性：访问控制模块自身安全
  - 行为正确性：按照安全策略验证



文件的保密

- 安全策略是一组明确的，经过良好定义的规则，描述访问行为的具体情况（如主体、客体、属性 等）与决策之间的对应关系。
- 安全策略主要分为两类：
  - 强制访问策略（MAC）：系统为每个主体和客体指派全局性的安全级别，访问请求发生时，通过比较安全级别来得出决策。指派的安全级别不可轻易修改。
  - 自主访问策略（DAC）：在主体与客体之间建立一组一致性原则，其中描述每个主体与每个客体之间的访问规则。这些访问规则由客体的创建者自主定义。

文件的保密

- DAC策略最常见的是基于主体权限的存取控制方式。其实现可采用三种数据结构：
  - 存取控制矩阵(PM)
  - 存取控制表(ACL)
  - 权能表(CL)

	$file_1$	$file_2$	$file_3$	$file_4$
u_1	r		rx	rw
u_2		wx		r
u_3	rx	w	x	rwx

